AI Engineering and Claude Architect · Part IV, Advanced

# Advanced AI Topics, Lesson 4

## Autonomy: Code Execution and Computer Use

An agent with 99% accuracy per step gets a 200-step task entirely right 13% of the time. Autonomy isn't a setting you turn up. It's a budget, spent by checkpoints, reversibility and containment.

| | |
|---|---|
| Module | AI11 of 13 |
| Lesson | 4 of 5 |
| Prerequisites | AI06 L7–L8, AI09 L2–L3, AI11 L3 |
| Exam relevance | Computer use isn't in the guide. Supports TS 1.4, 1.5, 5.2 |
| Reading time | ~30 minutes |

# Contents

1. Autonomy: Code Execution and Computer Use

*Lesson 4 of 5 · Advanced AI Topics*

# Autonomy: Code Execution and Computer Use

### *Key idea*

**Grant autonomy by how reversible an action is, and contain everything else.** Long runs compound small errors, so they need checkpoints and verification. Code and desktops should run somewhere disposable, with limits on time, output, network and credentials, as an agent's process can outlive the one you killed. Oversight means being able to intervene, and it doesn't mean approving every step.

> *Before: long-running harnesses, hooks, injection and sandboxes. Teaches: compounding error, deciding by reversibility, Claude's code execution and computer use tools, process containment, and what Anthropic measured on real autonomy. Used by: AI12.*

## The hook: small errors, long runs

Real output from `AutonomyLab.java`:

```
Chance that every step of a run is right, if failures are independent:
  each step 99.9%:    5 steps  99.5%   20 steps  98.0%   50 steps  95.1%  200 steps  81.9%
  each step 99.0%:    5 steps  95.1%   20 steps  81.8%   50 steps  60.5%  200 steps  13.4%
  each step 95.0%:    5 steps  77.4%   20 steps  35.8%   50 steps   7.7%  200 steps   0.0%
```

This is simple arithmetic, and it overstates the problem in one way, as agents often notice and fix mistakes. It also understates it in another: an undetected early mistake can spoil everything after it. Either way, **a long run with no checks is a bet against compounding.** The answer is checking work as you go, and keeping most actions reversible.

## Intuition: the new driver

A learner starts with a supervisor who has dual controls. Later they drive alone, on quiet roads. Later still, on motorways. Nobody decides this by asking how confident the learner feels. It depends on the track record, how bad a crash would be, and whether someone can grab the wheel.

The supervisor doesn't approve each turn of the steering wheel. They watch, and they're able to intervene. That's the distinction this lesson's research supports.

## Mechanism

### Decide by reversibility

| Reversibility | Examples | Default |
|---|---|---|
| No effect | Read, search, calculate | Autonomous |
| Easily undone | Edit a file in a branch, draft | Autonomous, logged |
| Costly to undo | Merge, bulk update | Checkpoint or review |
| Irreversible, or affects others | Payment, deletion, external send | Human, or strict limits |

A real policy adds **context:** has the agent read untrusted content in this run (AI09 L2)? Real output, from a sample policy I wrote:

```
A sample policy by action:
  read                          AUTO
  write, after untrusted input  AUTO, logged and undoable
  send, clean context           AUTO, allowlisted recipients only
  send, after untrusted input   ASK
  pay $20, clean context        AUTO, within budget
  pay $20, after untrusted      ASK
  pay $500                      ASK
  delete                        ASK
```

The code is a small switch, and it runs outside the model (TS 1.4, 1.5):

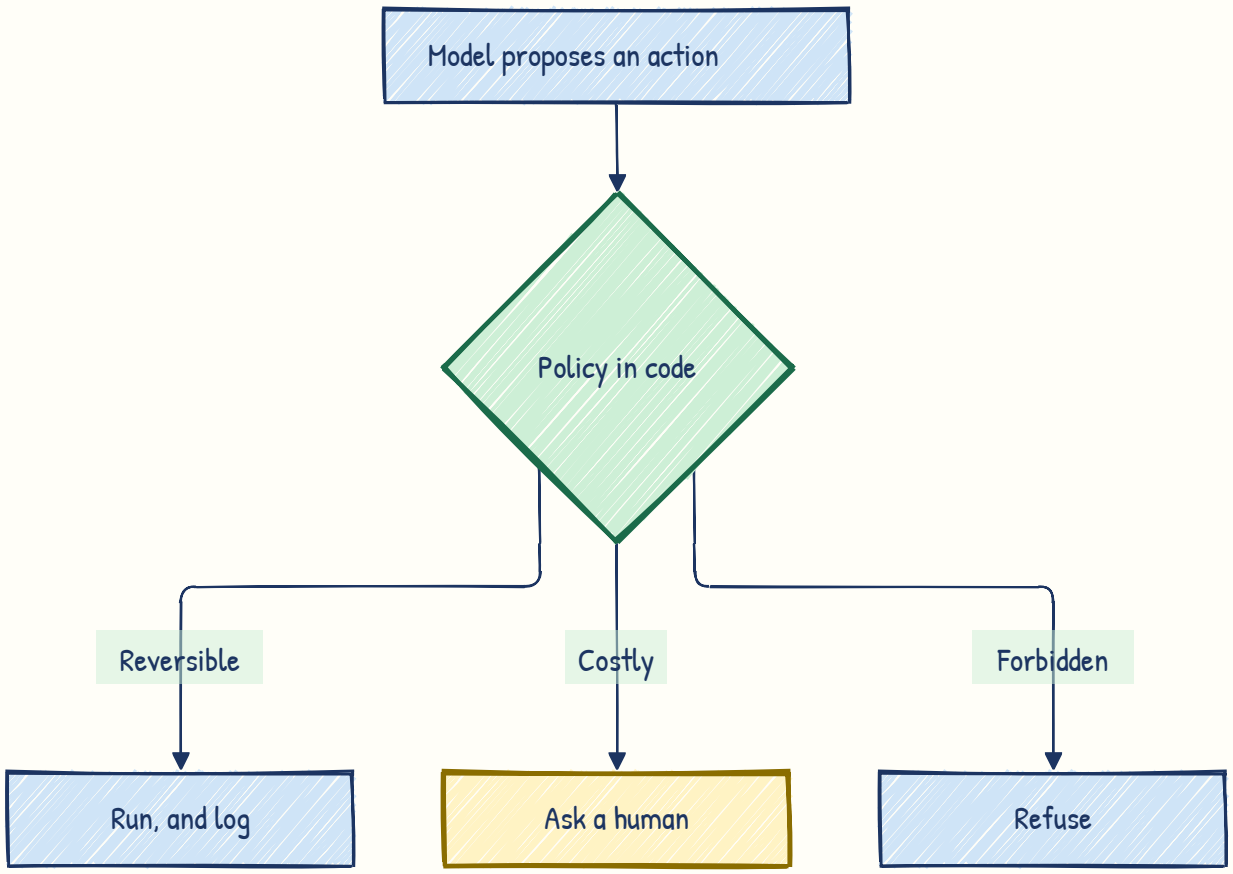



*Figure 1. The model proposes. Code decides by reversibility and context. Most low-risk work never reaches a person.*

## What Anthropic measured

Measuring AI agent autonomy in practice (February 2026) analyzed millions of interactions with Claude Code and the public API. It's the best evidence I've read for how oversight actually changes.

| Finding | Detail |
| --- | --- |
| Long turns are getting longer | 99.9th percentile rose from under 25 to over 45 minutes, October 2025 to January 2026. The median is about 45 seconds |
| Experienced users auto-approve more | About 20% of sessions for new users, over 40% by 750 sessions |
| **They also interrupt more** | About 5% of turns at around 10 sessions, about 9% for experienced users |
| The model stops itself | On the most complex tasks, it asks for clarification more than twice as often as on minimal ones |
| Most actions are low risk | About 80% of API tool calls have a safeguard, about 73% some human involvement, and 0.8% appear irreversible |

Experienced users shift from approving in advance to watching and intervening. The authors put it as: "effective oversight doesn't require approving every action but being in a position to intervene when it matters." They recommend that product developers give users clear visibility and simple ways to intervene.

**Keep the limits in mind:** this describes Claude Code and API usage in that period, with Anthropic's own measuring tool. Your users and domain may differ. Those figures say that irreversible actions are rare, and not that they're safe.

## Option 1: Anthropic's code execution tool

For running code without hosting anything. From the docs:

| Property | Value |
|---|---|
| Runs | Python and Bash in a sandboxed container, on Anthropic's side |
| Internet | **Completely disabled**, so no installing packages at runtime |
| Resources | 5 GiB RAM, 5 GiB disk, 1 CPU, and a maximum execution time |
| Isolation | From the host and other containers. File access limited to the workspace |
| Lifetime | Reuse by container ID. Expires 30 days after creation |
| Retention | **Not ZDR-eligible.** Container data retained up to 30 days |
| Cost | Free when used with current web search or fetch tools, and otherwise billed by execution time |
| Tool versions | Several, with programmatic tool calling in the later ones (AI06 L4) |

No internet removes an entire class of exfiltration, and it limits what the tool can do: it can't reach your database. For that, use your own tools.

## Option 2: computer use

Claude controls a desktop through screenshots, mouse and keyboard. From the docs:

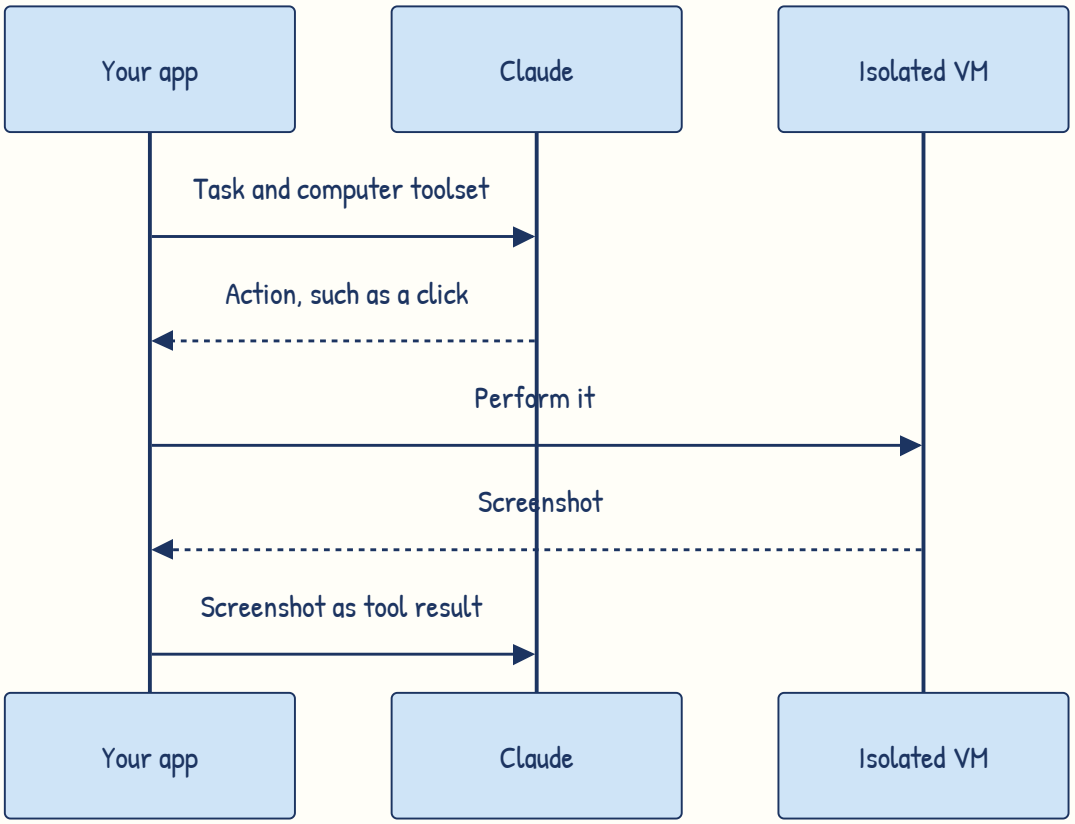



*Figure 2. Your application takes each action, in an environment you control. Claude only sees what the screenshots show.*

| Property | Detail |
|---|---|
| Tool | `computer_toolset_20260801` on 5.5 models, with 17 member tools and no beta header |
| Where it runs | **Client-side.** Your application runs every action, in your environment |
| Injection | Classifiers scan returned screenshots for potential prompt injection |
| Availability | Not currently available in Managed Agents |
| Web pages only | The docs recommend the browser use tool instead |
| Batches | Actions run in order, and stop at the first failure |

The docs' security recommendations are specific, and worth following exactly:

| Recommendation | Why |
| --- | --- |
| "Using a dedicated virtual machine or container with minimal privileges" | Limits accidents and attacks |
| "Avoiding giving the model access to sensitive data, such as account login information" | Prevents theft |
| "Limiting internet access to an allowlist of domains" | Reduces exposure to malicious content |
| "Asking a human to confirm decisions that might result in meaningful real-world consequences" | Payments, terms of service and cookies are named |

Anthropic's browser figures from AI09 L2 apply here: attack rates are low with safeguards, and they aren't zero. Prefer an API over a screen where one exists. A dedicated tool is checkable, whereas a click on coordinates isn't.

## Option 3: your own containment

If you run commands yourself, you need limits. I ran real child processes. **This controls time and output, and it's no sandbox.** They can still read files and use the network, and AI09 L3 covers that.

```
Real child processes, 1 to 5 second limits, 100,000 byte output cap:
  normal command   finished       kept       7 bytes
  endless loop     TIME LIMIT     kept       0 bytes
  output flood     OUTPUT LIMIT   kept 100,000 bytes
```

The output cap protects your memory, and your context. A flood passed straight to the model costs tokens at every later turn.

Here's the finding that surprised me. A parent process starts a child, and is then killed:

```
A parent starts a child that writes a file 3 seconds later:
  control, nothing killed: child wrote its file = true
  kill only the parent at 1.5 s: child wrote its file = true
  kill the process tree at 1.5 s: child wrote its file = false
```

**Killing the process you started doesn't stop what it started.** A background server or a runaway script keeps running after your "timeout." Kill the tree, and in production use a container or VM that you destroy whole. This ran on Windows, and behaviour differs by OS, so test yours.

## Keeping long runs honest

AI06 L7 covered long-running harnesses. The autonomy angle:

| Practice | Effect on compounding |
|---|---|
| Run tests after each change | Catches errors early, at the step they happened |
| Small checkpoints, such as commits | Limits how far back a mistake reaches |
| Verify outcomes in code | Doesn't rely on the agent's report (L3) |
| Budgets for steps, tokens and money | Stops a run going wrong for hours |
| Stop on repeated failure | Avoids looping (TS 5.2) |
| Independent review pass | A new context catches what the first missed (TS 4.6) |

## Earning more autonomy

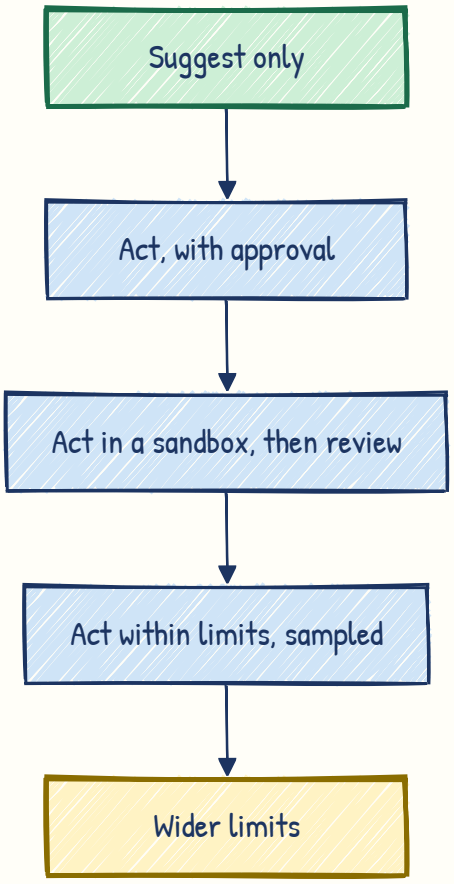



*Figure 3. Autonomy grows with evidence, and shrinks when an incident shows it should. Each step is reversible.*

Move up only with data: override rates, escapes and incidents. AI10 L3's sampling applies. Anthropic's study describes a "significant deployment overhang" between what models can handle and what they're allowed to do. So starting low isn't permanent, and it's still where to start.

## Why the obvious fixes fall short

| Obvious idea | Why it falls short |
|---|---|
| Approve every step | Fatigue. Experienced users move away from it |
| Trust the agent's report | Explanations aren't logs (L3) |
| Time out the process | Its children survive |
| Computer use for everything | Screens are unverifiable, and less reliable than an API |
| Give it your logged-in browser | Docs say to avoid sensitive data |
| A longer task, in one go | Compounding error |
| Permanent low autonomy | Costs value. Grow it by evidence |

## How Anthropic solved it

| Mechanism | What's published | Source |
|---|---|---|
| Code execution | Sandboxed, no internet | Tool docs |
| Computer use | Client-side, with injection classifiers | Tool docs |
| Security guidance | VM, no sensitive data, allowlist, human confirmation | Tool docs |
| Measuring autonomy | Post-deployment study | Research article |
| Claude Code | Modes, sandbox and classifier | AI09 L3, AI12 |
| Harnesses | Long-running agent patterns | AI06 L7 |
| Managed Agents | Hosted sandbox with permission policies | AI06 L8 |

# The production architecture

This is a recommended reference design, not a specific company's system.

**Baseline:** an agent running commands on a server.

**Improved:** a container, with timeouts. New problem: it holds credentials, can reach the network, and nobody watches what it does for hours.

**Production:** a disposable environment, a policy in code, and a live view with a stop button.

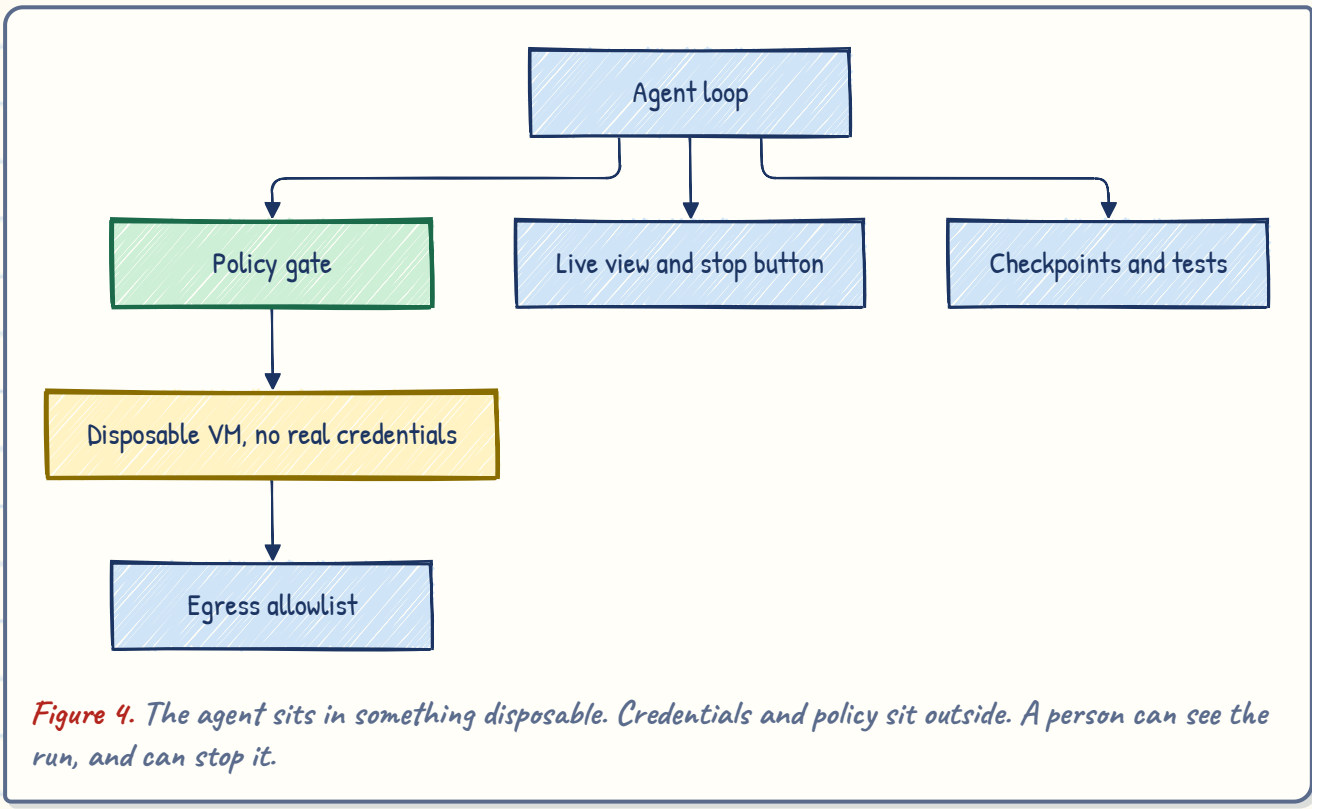



*Figure 4. The agent sits in something disposable. Credentials and policy sit outside. A person can see the run, and can stop it.*

| Component | Why it exists | How it communicates |
|---|---|---|
| Disposable VM | Destroyed whole, so nothing lingers | Orchestrator API |
| Policy gate | Reversibility rules | Hook, or tool layer |
| Egress allowlist | Limits exfiltration and exposure | Proxy |
| Credential proxy | No secrets inside | AI09 L3 |
| Budgets | Steps, tokens, time and money | Loop checks |
| Live view | Lets a person see and intervene | Event stream |
| Checkpoints | Bound the damage | Commits or snapshots |
| Audit | Reconstructs what happened | Ledger and recordings, with privacy care |

**Failures:**

| Failure | How you'd notice | What to do |
|---|---|---|
| Runaway process | Time limit | Destroy the VM |
| Output flood | Output cap | Truncate, and tell the agent it was cut |
| Injection on screen | Classifier flag, or odd action | Pause and ask. Limit authority |
| Loop with no progress | Step budget, repeated actions | Stop and escalate |
| Wrong click | Screenshot check, or a test | Verify state, and use APIs where possible |
| Undone work is missed | Review of reversals | Checkpoints |
| Long-run cost | Spend alert | Budgets |

## Java

Destroying the tree is one line, and it's the one most people omit:

```
process.descendants().forEach(ProcessHandle::destroyForcibly);
```

`descendants` must run before the parent dies, or there's no tree left to find. A process that has already detached can still escape it, which is one more reason for a VM.

The policy is an exhaustive `switch`, so a new action type won't compile until someone decides:

```
case PAY -> amount <= 50 && !sawUntrusted ? "AUTO, within budget" : "ASK";
```

**I haven't run computer use or code execution, as no API key is available.** Those facts come from the docs.

## Trade-offs

| Decision | Benefit | Cost |
|---|---|---|
| More autonomy | Speed and throughput | Compounding, and harder recovery |
| Provider's sandbox | Nothing to run | No internet, retention limits |
| Own VMs | Full control | Operations |
| Computer use | Works where no API exists | Slower, costlier and less checkable |
| Live oversight | Intervene when it matters | Someone has to watch |
| Frequent tests | Catch errors early | Time and tokens |

**When not to use computer use:** when any API or a browser tool can do the job. It's for the legacy system with no other interface.

### *Common mistakes*

**Treating a timeout as containment.** Children survive, and files and network remain reachable. Use a VM.

### Common mistakes

**Passing the agent your real browser session.** A page's hidden text can then act with your identity. Use a clean profile, and no stored logins.

### What to say in an interview

"Autonomy depends on reversibility and evidence. Errors compound over long runs, so I add tests and checkpoints throughout. A code policy allows reversible actions, asks on costly ones, and tightens after untrusted input. Execution happens somewhere disposable, with no credentials and an egress allowlist, because killing a process doesn't kill its children. Anthropic's data suggests oversight works by being able to intervene, so I'd build a live view with a stop button."

## Certification lens

### How this lesson maps to CCAR-F

**Computer use and code execution internals aren't in the guide.** Relevant: programmatic gates (TS 1.4), hooks (TS 1.5), escalation triggers such as no progress (TS 5.2), and independent review (TS 4.6).

These are original practice questions, not real or leaked.

**Q** Q1 · Exam style, scenario: Customer Support · Medium. An agent can issue refunds. Policy says that refunds above $500 need a manager, with no exceptions. What's the right implementation?

A. A strongly worded system prompt rule

B. An example in few-shot prompts

C. A hook or gate that intercepts the refund call in code, and redirects large amounts to a human

D. A daily report of large refunds, reviewed the next morning

**A** C (TS 1.4, 1.5). A guarantee needs enforcement outside the model. A and B improve probability only, and D detects it after the money has gone.

**Q** Q2 · An agent's steps are each 99% right. How would you make a 200-step migration practical?

**A** Without checks, the whole run is right 13.4% of the time, under independence. So split it into small batches. After each one, run tests in code, commit a checkpoint, and stop on repeated failure. Errors are then caught at their step, and only a batch is lost. Add an independent review at the end.

**Q** Q3 · Your tool's 60-second timeout fires, and yet a server the agent started keeps running. Why, and what's the fix?

**A** Killing a process doesn't stop its children, as my lab showed. Kill the process tree, and better, run in a container or VM you destroy whole, since a detached process can escape tree-killing. Also add limits on output, network and credentials.

## Revision - Lesson 4

→ Errors compound: 99% per step gives 13.4% for 200 steps, if independent.

→ Decide by reversibility and context, in code. Tighten after untrusted input.

→ Anthropic's study: experienced users auto-approve more, and interrupt more. 0.8% of calls appear irreversible.

→ Oversight means being able to intervene, not approving each step.

→ Code execution tool: no internet, 5 GiB RAM and disk, 1 CPU. Not ZDR-eligible.

→ Computer use: runs client-side. Classifiers scan screenshots. Not in Managed Agents.

→ Docs advise: VM, no sensitive data, domain allowlist, human confirmation.

→ Prefer an API over a screen.

→ Killing a parent leaves its children. Kill the tree, or destroy the VM.

→ Use tests, checkpoints and budgets throughout. Grow autonomy with evidence.

*Your notes*